COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	OCTOBER 13 2026	**DATE SUBMITTED:**	OCTOBER 15 2026
NAME:	JARED ISAIAH P. DELA CRUZ DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 12.1 **Hands-on Activity 12.1 Algorithmic Strategies**

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Quick Sort	It is a recursion strategy because it chooses a pivot, partitioning the array, and making recursive function calls on the smaller sub arrays until it reaches the base case.
Brute Force	Linear Search	Linear search sequentially checks every single element in the array from the start up to the end. It will only stop if the target is found or all elements have been checked.
Backtracking	Quick Sort	In partitioning the array, it will pick an element and swaps based on the comparison to the pivot. If a condition is not met for an element, the pointers backtrack to try a different element placement.
Greedy	Selection Sort	The algorithm greedily makes the locally optimal choice at each iteration by checking the remaining unsorted portion so that it can find the absolute minimum element.
Divide-and-Conquer	Merge Sort	In this algorithm, it divides the unsorted array into two equal

		halves, it also recursively sorts each half, and then combines the sorted sub arrays back together into a single sorted array.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	Code:
	`#include <iostream>`
	
	`//Memoization table 12-2`
	
	`int memo[101]; // we will initialize the elements to -1 ( -1 means, not solved it yet )`
	`int getMinSteps ( int n )`
	`{`
	`  if ( n == 1 ) return 0; // base case`
	`  if( memo[n] != -1 ) return memo[n]; // we have solved it already :)`
	`  int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the optimal answer finally`
	`  if( n%2 == 0 ) r = std::min( r , 1 + getMinSteps( n / 2 ) ) ; // '/2' step`
	`  if( n%3 == 0 ) r = std::min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step`
	`  memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.`
	`  return r;`
	`}`
	
	`int main() {`
	
	`  //memoization`
	`  int testCases[] = {1, 4, 7, 10};`
	
	`  for (int n : testCases) {`
	`  // Reset memo array to -1 before each test case`
	`  for (int i = 0; i <= n; i++) {`
	`  memo[i] = -1;`
	`  }`
	
	`  std::cout << "n = " << n << " - Min Steps: " << getMinSteps(n) << std::endl;`
	
	`  }`
	
	`  return 0;`
	`}`
	
	Output:
	`n = 1 - Min Steps: 0`
	`n = 4 - Min Steps: 2`
	`n = 7 - Min Steps: 3`
	`n = 10 - Min Steps: 3`
	
	`[Done] exited with code=0 in 0.521 seconds`

Analysis	**Dela Cruz:** From a theoretical perspective, this code solves the minimum steps problem using a top-down dynamic programming approach that relies on optimal substructure and overlapping subproblems. The problem breaks down into finding the minimum result among three possible transitions, which are subtracting one, dividing by two, or dividing by three, which allows the

	optimal answer for nnn to be derived directly from the optimal answers of smaller numbers. By storing each calculated result in a global array initialized with a placeholder value (−1-1−1), the algorithm uses memoization to eliminate redundant recursive calculations. This technique optimizes the worst-case time complexity from an exponential O(3^n) down to a linear O(n), as each sub-state from 111 to nnn is evaluated only once in constant time, while requiring O(n) auxiliary space to store the memoization table and manage the call stack depth. **Mapas:**In a theoretical analysis, memoization slashes execution time from exponential O(3^n) down to a linear O(n) by saving every step in an array to avoid repetition of work. And in terms of the experimental approach, it produced all the expected minimum steps for all inputs.

Table 12-2. Memoization Implementation

Screenshot	Code:
	``cpp
	#include <iostream>
	
	//Bottom-Up DP table 12-3
	
	int getMinSteps (int n)
	{
	    int dp[n+1], i;
	    dp[1] = 0; // trivial case
	    for(i = 2 ; i <= n ; i ++)
	    {
	        dp[i] = 1 + dp[i-1];
	        if(i%2==0) dp[i] = std::min(dp[i] , 1+ dp[i/2]);
	        if(i%3==0) dp[i] = std::min(dp[i] , 1+ dp[i/3]);
	    }
	    return dp[n];
	}
	
	int main() {
	
	    int testCases[] = {1, 4, 7, 10};
	
	    // bottom up DP
	
	    for (int n : testCases) {
	        std::cout << "n = " << n << " - Min Steps: " << getMinSteps(n) << std::endl;
	
	    }
	
	    return 0;
	}
	
	``
	
	Output:

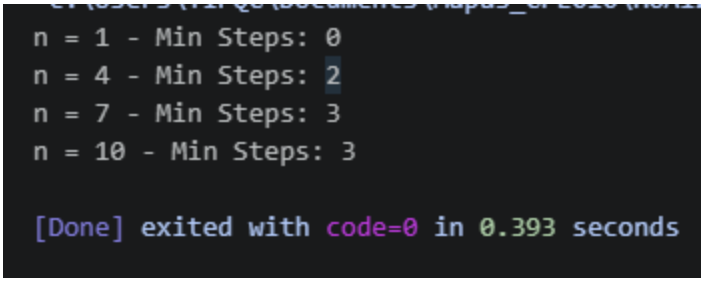
	
Analysis	**Dela Cruz:** This code calculates the minimum number of steps needed to reduce a given number n down to 1 using three possible operations: subtracting 1, dividing by 2, or dividing by 3. Starting from 1, which takes 0 steps, it fills an array step-by-step from 2 up to n, finding the best choice for each number by comparing the steps required for all valid operations. By building the answer from the ground up, it avoids repeating calculations and efficiently returns the smallest number of steps for each test input. **Mapas:** Theoretically analysing this, bottom up dynamic programming makes solutions sequentially 0 up to n. So that it will yield an O(n) optimal time and space efficiency. And in terms of an experimental approach the output matched the memoization approach perfectly, and it also has a faster execution time.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

CODE SNIPPET:

```
    int matrix[4][4] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost = 21;
    std::cout << "\nNumber of valid paths: " << countPaths(matrix, 0, 0, targetCost) << '\n';

    return 0;
}
```

main.cpp function

```cpp
template <int M, int N>
int countPaths(int (&cost)[M][N], int i, int j, int remaining){
    if (i >= M || j >= N)
        return 0;

    remaining -= cost[i][j];

    if (remaining < 0)
        return 0;

    if (i == M - 1 && j == N - 1) {
        return (remaining == 0) ? 1 : 0;
    }

    return countPaths(cost, i, j+1, remaining) + countPaths(cost, i+1, j, remaining);
}

```

Header file

```
Valid Paths Available: 1
```

Output

PSEUDOCODE:

countPaths(matrix, row, col, remainingCost):

if the initial position is out of bounds:

return 0

Each function call:

subtract the cost of the current cell from remainingCost

If remainingCost is negative:

return 0

If we have reached the last cell:

if remainingCost is 0:

return 1

else:

return 0

return countPaths(matrix, row, col + 1, remainingCost) + countPaths(matrix, row + 1, col, remainingCost)

***/**

CODE ANALYSIS

Dela Cruz: For this program, it utilizes recursion to explore every possible path through the grid starting from the top-left cell and moving only down or right. As it visits each cell, it subtracts that cell's value from the total target cost. If it reaches the bottom-right destination cell with a remaining cost of zero, it counts that as a valid path. Finally, it adds up all successful paths from both direction choices and returns the total count.

Mapas: In this program, the code uses recursion to find the total number of paths from the top left to the bottom right corner of the matrix. It moves right or down across cells, subtracting each cell value from a starting target cost of 21. Lastly a path is counted as valid only if it reaches the final destination with the remaining cost reduced to exactly zero.

CONCLUSION

Dela Cruz

- Overall, the activity helped me demonstrate how dynamic programming and backtracking effectively tackle complex optimization and counting problems by breaking them into smaller, manageable subproblems. Whether using a top-down recursive approach with memoization or a bottom-up iterative table, caching overlapping subproblems prevents redundant work and dramatically speeds up calculation time compared to plain recursion. Meanwhile, standard recursive traversal explores every path step-by-step, making it ideal for counting valid grid routes under specific target constraints. Together, these fundamental techniques highlight how storing past results and structuring decision trees turn slow, exponential algorithms into fast, efficient solutions.

Mapas

- With that, I can conclude that this activity helped me to demonstrate and understand algorithmic strategies. I also created C++ codes to implement various algorithmic strategies, and to solve problems using different algorithmic strategies. Furthermore I also did analysis of every task in the procedure and supplementary. I also realized that there are a lot of algorithmic strategies, and it is best to use what is fitting to the program or problem being solved. Lastly, I still have a lot to understand and I will keep learning to enhance my programming skill.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used AI as a tool of understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.